COMP1531
Software Engineering Fundamentals

Week 5
Full-stack - HTTP Servers & Testing

Lecture Objectives:

– – –

- Recap
- Server Testing and Requests Made Via Code
- HTTP Status Codes
- Swagger Files
- And finally:
    - **Iteration 2**

Recap: How Servers Receive Information

Method	Visible in Logs	Cached	Browser History	Encrypted by HTTPS	Safe for Passwords
URL (query string)	✅	✅	✅	✅	❌
POST body	❌	❌	❌	✅	✅

Recap: What HTTPS does:

– – –

HTTPS encrypts all data sent between the client and server

- URL path (/users/login)
- Request headers
- Request body (for POST, PUT, etc.)
- Query parameters in the URL

Recap: CRUD

Recap: How to Use CRUD Operations in Your HTTP Requests

---

1. Locally you can install "**Advanced Rest Client**" or on the CSE machines you can run 1531 arc.

   OR

2. Use an npm package **sync-request-curl** to allow us to programmatically send RESTful API requests.

Requests Made Via Code

We **don't** need an API client to send requests to servers, we can actually send requests via code

Testing HTTP Requests Without an API Client

- Direct code-based requests are **easy to automate**.
- This makes them a great fit for **Vitest integration and API testing**.

Requests Library

We can use an npm package `sync-request-curl` to allow us to programatically send RESTful API requests.

```
npm install --save-dev sync-request-curl
```

We can send HTTP requests to our server

Requests Library

```
import request from 'sync-request-curl';

const res = request(
  'POST',
  'http://localhost:3000/books',
  {
    json: {
      title: 'helloeveryone',
      author: 1531,
      year: 2025
    },
  }
);
const res1 = request(
  'GET',
  'http://localhost:3000/books',
);

console.log(JSON.parse(String(res.getBody())));
```

Working with **Vitest**

Step 1: Install Vitest + dependencies (Refer to Lecture 3.1)

Step 2: Install @vitest/coverage-v8 (Refer to Lecture 3.1)

```
1   import request from 'sync-request-curl';
2   import {describe, test, expect} from 'vitest'
3
4
5   describe ( 'Testing server endpoint', () => {
6
7       test ('Testing /hello/:name/:age endpoint', () => {
8           const res = request('GET', 'http://localhost:3000/hello/COMP1531/4')
9           const bodyObj = String(res.body);
10          expect(bodyObj).toBe(JSON.stringify ({ name: 'Hello COMP1531', age: 'Your age is 4' }));
11      })
12
```

express_server.tests.js

How can we run the Jest test runner on our test file?

```
npm run test express_server.test.js
```

HTTP Response Status

Every HTTP response from a server comes with a status code (e.g. 200 OK, 404 Not Found).

It tells the client what happened:

2xx → success
4xx → client error
5xx → server error

Problem

What endpoints are available?

What parameters should I send?

What status codes might I receive?

What does a successful response look like?

Solution: Formal, machine-readable API contracts → Swagger (OpenAPI) file

Swagger File

Swagger makes it explicit.

A Swagger file is a YAML/JSON document that:

- Lists API endpoints
- Defines expected parameters
- Specifies possible response status codes
- Describes the response body structure

Online editor: https://editor.swagger.io/

Set up and Serve interactive API documentation (Swagger UI)

npm install swagger-ui-express yamljs

Example:

```
import express from 'express';
import swaggerUi from 'swagger-ui-express';
import YAML from 'yamljs';

const app = express();
const swaggerDocument = YAML.load('./swagger.yaml');

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));

app.listen(3000, () => console.log('Swagger docs running at http://localhost:3000/api-docs'));
```

Summary

- We can test servers through code
- HTTP status codes are 3-digit numbers a server returns in response to a client request.
- They help the client understand whether the request was successful or if something went wrong.
- Swagger (now called OpenAPI) is a standard for describing RESTful APIs in a machine- and human-readable format, usually written in YAML or JSON

Feedback

COMP1531 Feedback Survey